

InMobi Click-a-thon 2026 · Track: InMobi

RCA-Mole

Digging into the root cause of event-stream anomalies, automatically, and grounded in real numbers.

Team Columnar Knights

M P Shri Veena · Pranav Gopalkrishna · Sandesh Aithal · Shivaprakash Akki

WHAT IT DOES

Find the anomaly. Find the cause. Explain it in plain English.

1

Detect

Flag when a key ad-metric (revenue, fill rate, eCPM, requests, CTR) deviates from its like-for-like baseline.

2

Attribute

Decompose the metric and rank every segment (device, region, publisher, format...) by real explanatory power.

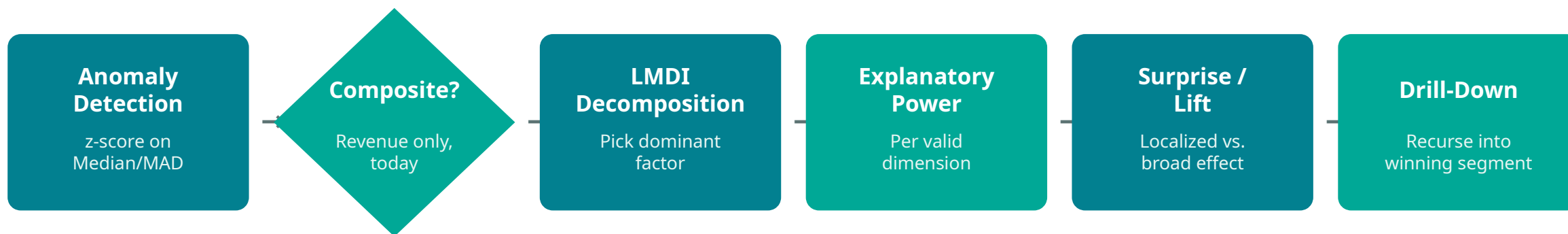
3

Narrate

An LLM turns the computed JSON into a short, grounded diagnosis; every number traceable back to ClickHouse.

Every step, every SQL query and the one LLM call, is captured in a local JSON trace and mirrored to Langfuse, so a judge can see exactly what was checked, in what order, and why.

One pipeline: detection → attribution → narrative



Atomic metrics (fill rate, eCPM, CTR, requests) skip the LMDI step entirely and go straight to Explanatory Power / Lift / Drill-Down.

CLI

rca scan | investigate | auto: JSON + trace output for every run.

Dashboard

Live metric charts, anomaly markers, incident list, full drill-down reports.

Chat (LibreChat)

Ask in natural language; the model calls the same pipeline as a tool.

Five layers, one grounded pipeline

1. Algorithmic Workflow

Anomaly detection (z-score/MAD + trend+seasonality baseline) → LMDI decomposition for composite metrics → Adtributor-inspired attribution (Explanatory Power + Lift).

2. Core Analytical Codebase

Python, orchestrating ClickHouse SQL for every stage of the workflow.

3. ClickHouse Data Layer

Dimension tables + event table, pre-joined into a single denormalized fact table (ClickHouse Cloud).

4. UI Layer

Locally-hosted HTML/JS/CSS: dashboards, investigation overviews, drill-down trees, chat UI link.

5. AI Layer

LibreChat (public) + Gemini, with investigation context injected on startup, chat history persisted in MongoDB, and Langfuse monitoring latency, traces, tokens, and errors.

z-Score on Median/MAD: a baseline that isn't fooled by noise

Baseline = Trend + Seasonality

Trend:	Linear regression over previous seasonal points; nets out slow growth.
Seasonality:	Repeating shape tied to a calendar unit (e.g. day-of-week).
Residual:	actual — baseline. This is what's actually checked for anomalies.

$$\text{z-score} = (\text{actual} - \text{median baseline}) / (1.4826 \times \text{MAD})$$

MAD (median absolute deviation) is used instead of mean/std-dev because it's insensitive to the very anomalies we're trying to detect.

Key Tunable Variables

- **Aggregation interval**
Time bucket for computing metrics
- **Seasonality interval**
Repeating rhythm to account for
- **Deviation threshold**
How much deviation counts as anomalous
- **Recency window**
How many past seasonal points feed the trend

LMDI: which factor moved, exactly, no guessing

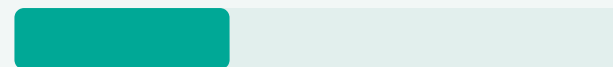
$$\text{Revenue} \approx \text{Requests} \times \text{Fill Rate} \times \text{eCPM} / 1000$$

Naive approaches to splitting credit between multiplied factors either misjudge magnitude (change-one-at-a-time, revert) or depend on the order you change factors in (change-one-at-a-time, keep). LMDI solves both: it uses a log-mean weighting so factor contributions add up exactly to the observed delta, independent of order.

- Factor contributions sum exactly to the observed revenue delta
- Order-independent, unlike naive step-by-step attribution
- Implemented in `decompose_revenue()` in `attribution.py`

Revenue Delta, Explained

Requests



Fill Rate



eCPM



*(illustrative; real contributions come from
decompose_revenue() on ClickHouse data)*

Explanatory Power: how much of the movement is this segment's fault?

Additive metrics

Requests, Revenue

EP is just the segment's share of the total delta: direct and simple.

$$EP(\text{segment}) = \Delta\text{segment} / \Delta\text{total}$$

Example: Requests drop 200K overall. NAM and EU each drop 100K
→ EP(NAM) = EP(EU) = 50%. All segment EPs sum to 100%.

Ratio metrics

Fill Rate, Render Rate, CTR, eCPM

A segment can drive the whole rate change through pure mix/volume shift, even if its own rate never moved. Naive rate-diffing misses this entirely.

Finite-difference decomposition isolates rate effect from volume effect

Example: EU's own fill-rate is unchanged, but EU's share of requests grew 30%→60%, and the blended rate drops purely from mix shift.

Surprise / Lift: separating real causes from “along for the ride”

$$\text{lift} = \text{Explanatory Power} / \text{expected_share}$$

$$\text{lift} \approx 1$$

Proportional, not a cause

The segment explained roughly as much of the movement as its size alone predicts. E.g. Data Center X carries 94% of revenue both before and after: high EP, but just “along for the ride.”

$$\text{lift} \gg 1$$

Genuine localized cause

Disproportionate to its size. E.g. Device Type: PC swells from 50% to 98% of revenue share while Mobile/Tablet collapse: a real, localized shift worth reporting.

STEP 4 · DRILL-DOWN

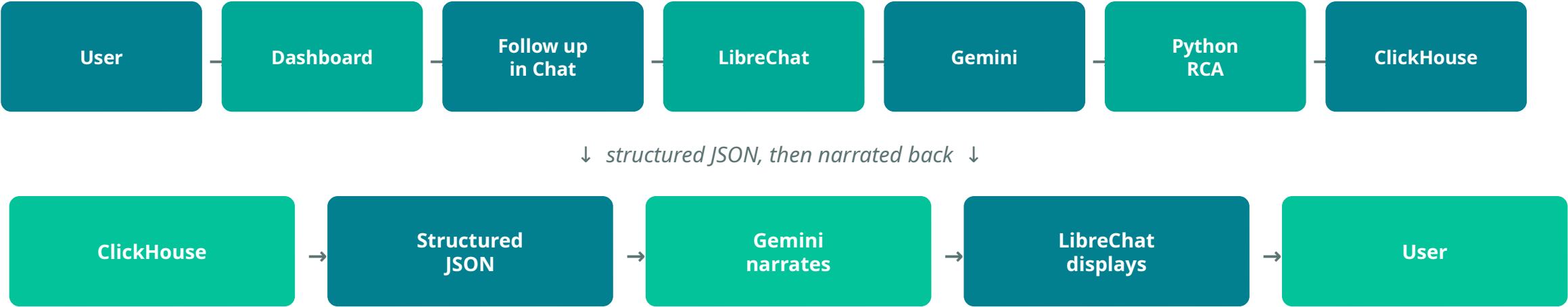
Recurring into the winning segment, with the right dimensions

Once a segment clears EP + lift thresholds (e.g. region = EU), drill-down re-scores the remaining dimensions inside that segment (e.g. region = EU AND device_model = iPhone 13), recursing until nothing clears the bar, or max depth is hit.

Metric	Population	Valid dimensions
Requests / Fill Rate	Pre-fill (all requests)	ad_format, category, publisher_tier, region, country, device_model, os_version
Render Rate / CTR / eCPM / Revenue	Post-fill (impressions)	All of the above, plus vertical, campaign_type

Why it matters: vertical/campaign_type only exist on filled requests (advertiser_id is empty pre-fill). Excluding them for Requests/Fill Rate isn't a post-hoc filter: it's structural, avoiding a fabricated "all-unfilled-traffic" segment with a rate fixed at 0.

From “why did this drop?” to a grounded answer



- Frontend:** Displays incidents; lets the user start a chat.
- Gemini:** Understands the question; explains RCA results in natural language.
- ClickHouse:** Stores incident, metric, and event data.
- Langfuse:** Monitors the AI pipeline: latency, traces, tokens, errors.

- LibreChat:** Chat interface for interacting with Gemini.
- Python RCA:** Performs the investigation; generates structured JSON.
- MongoDB:** Persists chat history and user sessions.

Thank You

RCA-Mole: grounded, traceable root-cause analysis for ad-metric anomalies.

- **Repo:** github.com/columnarknights/rca_mole
- **Docs:** [docs/demo-solution-approach-2.md](#)
- **Demo Video:** [linked separately](#) (2–3 min walkthrough)

Team Columnar Knights

M P Shri Veena · Pranav Gopalkrishna · Sandesh Aithal · Shivaprakash Akki

Track: InMobi · Project: RCA-Mole